



Brandenburg
University of Technology
Cottbus - Senftenberg

Brandenburgische Technische Universität
Cottbus–Senftenberg

Study Programme:

Master of Science in Artificial Intelligence

Information Retrieval

Practical Task No.1

Submitted by:

HAJJI Mohamed Amine
Rushikesh Suhas Wagh

Instructor:

Prof. Dr.Ingo Schmitt
Prof. Dr.George Assaf

Date:

May 9, 2026

1 Diagram

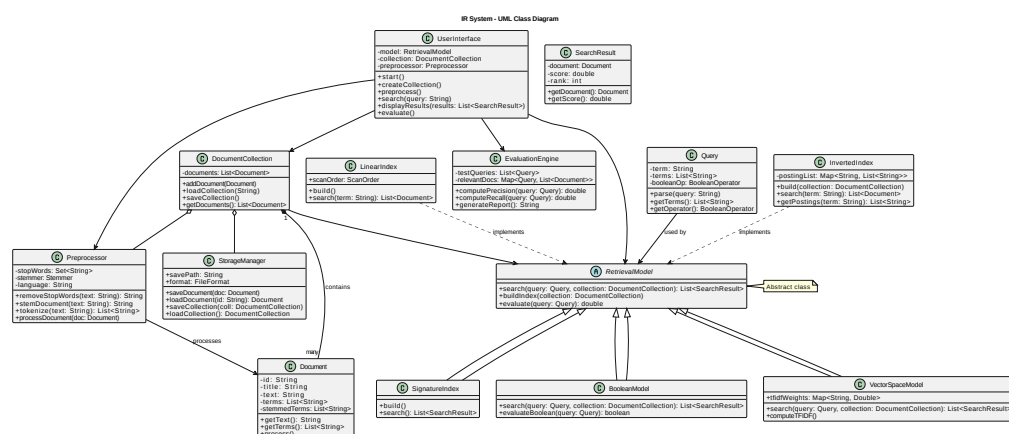


Figure 1: UML class diagram

2 Explanation of the UML Diagram and Relationships

The UML diagram represents an Information Retrieval (IR) system that implements all functionalities required in the task sheet. In total, it consists of thirteen classes, each representing a specific component of the system. The overall workflow follows the natural order of document processing: document creation, preprocessing, indexing, retrieval, evaluation, and user interaction.

The **Document** class is the fundamental entity of the system. It represents textual documents or web pages and contains attributes such as **id**, **title**, **text**, and **terms**. Its methods **getText()**, **getTerms()**, and **process()** allow access to the textual content, extraction of relevant terms, and preprocessing of the document.

The **DocumentCollection** class stores multiple instances of the **Document** class. It manages the corpus through methods for adding, loading, saving, and returning documents. This creates a one-to-many association between **DocumentCollection** and **Document**, since one collection consists of many documents.

The **StorageManager** class is responsible for storing documents on the hard disk and loading them after restarting the program. It manages the storage location and file format and can save or load both single documents and complete collections.

The **Preprocessor** class prepares documents for retrieval. It contains stop words and stemming algorithms as attributes and provides methods for tokenization, stop-word elimination, stemming, and a complete preprocessing pipeline. This class depends directly on the **Document** class, because it modifies and transforms the textual content of each document into a form suitable for retrieval.

The **Query** class handles user input. It converts a search query into a structured form by storing terms and Boolean operators. The methods of this class parse the query and extract the search terms and operators.

The **RetrievalModel** class is an abstract superclass that defines the general search interface. It cannot be instantiated directly but serves as the basis for concrete retrieval models. The two subclasses **BooleanModel** and **VectorSpaceModel** inherit from it. This inheritance relationship demonstrates abstraction and polymorphism in the system design.

The **BooleanModel** class implements Boolean retrieval logic, while the **VectorSpaceModel** class performs ranking-based retrieval. In the latter, document vectors are weighted using the **computeTFIDF()** method.

To support efficient searching, the system includes the indexing classes **InvertedIndex**, **LinearIndex**, and **SignatureIndex**. These classes build searchable structures from the document collection and provide different implementations of retrieval.

The **SearchResult** class represents the output of a search process. It stores matched documents together with their relevance scores and ranking information.

The **EvaluationEngine** class evaluates retrieval quality. Using predefined test queries and relevant document sets, it computes precision and recall, allowing the system to

measure search effectiveness.

Finally, the `UserInterface` class acts as the central coordination component. It connects the main classes of the system and allows the user to create document collections, preprocess documents, perform searches, display results, and evaluate system performance.

The arrows in the UML diagram describe the relationships between these classes. The `DocumentCollection` class has an association with the `Document` class. The `Preprocessor` depends on `Document`. The `Query` interacts with the abstract `RetrievalModel`, which is implemented by `BooleanModel` and `VectorSpaceModel`. The indexing classes depend on the document collection to build searchable structures. The `EvaluationEngine` depends on queries and relevant documents, while the `UserInterface` coordinates all major components of the system.

Overall, the UML diagram presents a modular architecture in which all classes collaborate to support document storage, preprocessing, indexing, searching, evaluation, and user interaction in a complete Information Retrieval system.